关卡编辑器新手教程

音乐练习 UGC 平台 · 由浅入深

2026 年 10 月 • 基于编辑器 v3（画布 + 数据表 + 节点图）架构撰写

本教程面向第一次接触本平台编辑器的音乐老师与内容创作者：**不需要任何编程基础**。全文按「由浅入深」排列——先认识平台与界面（第 1 章），搭出第一幅画面（第 2 章），填入题目内容（第 3 章），再学习用节点图给关卡加入逻辑（第 4 章），并用两个完整案例把前三章串起来（第 5 章）；第 6–7 章覆盖调试技巧、进阶控制流、脚本与 JSON 视图以及 AI 协作。各章末尾的小练习建议逐个完成，全部走完约需 60–90 分钟。

目录

1 认识平台与编辑器 3

1.1 这是一个什么平台 . . . 3
1.2 关卡由哪几部分组成（核心概念） . . . 3
1.3 从哪里进入编辑器 . . . 4
1.4 编辑器界面导览 . . . 4
1.5 上手前的建议：先玩两个内置关卡 . . . 5

2 画布：搭出你的第一个界面 5

2.1 组件面板：三种类型的元素 . . . 5
2.2 摆放与调整组件 . . . 6
2.3 属性检查器 . . . 6
2.4 视图：互斥的多幅画面 . . . 7
2.5 保存并试运行 . . . 7

3 数据表：填入题目内容 7

3.1 一行就是一道题 . . . 7
3.2 列与列类型 . . . 8
3.3 五线谱编辑器 . . . 8
3.4 推进与过关设置 . . . 9

4 节点图：让关卡活起来 **9**
4.1 工作台导览 9
4.2 事件：一切逻辑的起点 10
4.3 连节点：把流程画出来 10
4.4 动作节点：指挥组件做事 11
4.5 变量与计分 11
4.6 保存、检查、试运行 12

5 实战：两个完整案例 **12**
5.1 案例一：听音选择题（数据驱动） 12
5.2 案例二：多视图闯关流程（视图驱动） 13
5.3 两种形态怎么选 14

6 进阶技巧 **14**
6.1 条件、循环、等待与自定义事件 14
6.2 表达式速成 14
6.3 计时器与隐形组件 15
6.4 调试模式：看得见的排错 15
6.5 排错工具箱 15

7 高级：脚本、JSON 与 AI 协作 **15**
7.1 脚本页：用文字写逻辑 15
7.2 JSON 页：关卡的原始形态 16
7.3 AI 助手：你的创作副驾驶 16
7.4 导入、导出与分享 17

8 附录 **17**

1 认识平台与编辑器

1.1 这是一个什么平台

音乐练习 UGC 平台是一个在浏览器里运行的**音乐练习内容创作与游玩工具**：老师或创作者在**编辑器**里搭建练习关卡，学习者在**游玩页**里答题、听音、练节奏。所有内容保存在本地浏览器数据库中，也可以导出成文件分享给别人。

内容按三个层级组织：

层级	类比	说明
系列	一本教材	一个完整的课程包，可整体导出为.zip 分享
专题	一章	系列下的分组
关卡	一节练习课	真正可游玩的单元，也是编辑器编辑的对象

1.2 关卡由哪几部分组成（核心概念）

打开编辑器后，一个关卡被拆成五个可以分别编辑的组成部分。理解这张表，后面的所有操作都会有归属感：

组成部分	在哪个标签页编辑	回答的问题
视图（画面）	「**画布**」	玩家看到哪些画面？
组件	「**画布**」	画面上摆放哪些元素（按钮、五线谱……）？
数据表	「**数据表**」	题目内容是什么？
逻辑（节点图）	「**节点图**」	玩家操作之后发生什么？
推进设置	「**数据表**」（表格上方）	按什么顺序出题、怎样算过关？
变量	「**节点图**」（变量面板）	得分等状态记在哪里？

整个编辑器最重要的设计思想是一条黄金法则：

【黄金法则：改哪层，去哪层】

- 想**换内容**（换题目、换音符、改文案）——只改**数据表**，逻辑一行不动；
- 想**换画面**（挪按钮、加个标题）——只改**画布**；
- 想**换行为**（答对加几分、点按钮去哪）——只改**节点图**。

内置示例「音色实验室」最能体现这一点：想加第七种音色，只需在数据表**加一行**，逻辑零改动。

几个贯穿全文的名词，先混个脸熟即可，后文会反复解释：

- **组件实例**：拖到画布上的每一个具体元素（比如一个按钮），都有一个名字（如 `button1`）；
- **行**：数据表的每一行就是一道题；行号（从 0 开始）记在系统变量 `v.__row` 里；
- **q．列名**：在逻辑里引用「当前行某列」的内容，例如 `q.title`、`q.notes`；
- **v．变量名**：引用一个变量，例如默认就有的分数变量 `v.score`；
- **事件**：玩家操作或系统状态变化时发出的信号（如「答对了」「进入了视图」），是节点图里一切逻辑的起点。

1.3 从哪里进入编辑器

打开应用后，顶部导航栏有：「**首页**」、「**资源库**」、「**校音器**」、「**节奏**」、「**设置**」。编辑器从**资源库**进入，共四种方式：

1. **新建空白关卡**：资源库顶部点 「**新建关卡**」，会创建一个带默认结构的关卡并直接进入编辑器；
2. **编辑现有关卡**：任意关卡行点 「**编辑**」。编辑内置示例时不必有顾虑——首次保存会自动**另存为你的副本**，内置原版不会被覆盖；
3. **从游玩页进入**：试玩某关时点顶部的 「**编辑此关**」；
4. **AI 新建关卡**：点 「**AI 新建关卡**」，用一句自然语言描述想要的内容（如「大三和弦听辨练习，10 题，每题 10 分」），先建关卡再让 AI 助手继续完善（需要先在设置页配置 AI，见第 7 章）。

1.4 编辑器界面导览

进入编辑器后，界面分两层：**顶部工具栏** + **五个标签页**。

顶部工具栏从左到右：

元素	作用
「**返回资源库**」	回到资源库列表
标题输入框	修改关卡标题（保存在关卡信息里）
「**撤销**」/ 「**重做**」	快捷键 Ctrl+Z / Ctrl+Shift+Z（也支持 Ctrl+Y），最多 50 步
「**保存**」	保存当前修改；有未保存修改时按钮会带星号（「**保存 ***」）
「**试运行**」	先自动保存，然后跳到游玩页真实试玩本关

保存后界面上方可能出现三种颜色的提示条，含义固定：

- **红条**「无法保存，请修正：」——存在结构性错误（附错误清单），修正后才能保存；
- **黄条**「lint：…」——能保存，但存在可疑之处（如引用了不存在的列），建议处理；

- **绿条**「已保存（v 版本号）」——保存成功。

五个标签页即 1.2 节五层结构的入口：「**画布**」、「**节点图**」、「**脚本**」、「**数据表**」、「**JSON**」，标签栏最右侧还有一个「**AI**」按钮（AI 助手在所有标签页共享同一会话，见第 7 章）。「**脚本**」与 「**JSON**」是同一份逻辑的两种「文字形态」，属于进阶内容，新手可以先无视。

> **【离开页面的保护】**有未保存修改时切换到站内其他页面或关闭标签页，编辑器会弹窗确认，防止误丢内容。养成「改一段就点一次保存」的习惯最稳妥。

1.5 上手前的建议：先玩两个内置关卡

资源库顶部的「教程与示例」分区里，有两个专为新手准备的关卡：

1. **新手引导 · 交互教程**：一个 5 步的可玩教程，带你以玩家身份体验按钮点击、滑块拖动、听音选答和重开关卡——玩完它，你就知道玩家能做什么操作；
2. **音色实验室 · 合成器示例**：一条逻辑图驱动一张 6 行数据表。先玩一遍，再点 「**编辑**」看它的**数据表**：每种音色就是一行数据——这是「数据驱动」最好的示范。

> **【提示】**想偷看内置关卡的内部构造，可以打开它的 「**编辑**」，在 「**节点图**」和 「**数据表**」里随意浏览（不点保存就不会改动任何东西）；游玩页顶部的 「**逻辑图谱**」也能以只读方式查看整关的逻辑图与变量表。

2 画布：搭出你的第一个界面

画布标签页是三栏布局：**左侧组件面板**、**中间视图条 + 画布**、**右侧属性检查器**。

2.1 组件面板：三种类型的元素

左侧组件面板按用途分三组，点击某项即可把它添加到当前视图：

分组	组件	用途
音乐	五线谱	展示乐谱，可配置「音符可点击」
音乐	发声器	播放音符串（采样音色：钢琴、电钢、木琴等）
音乐	指法 / 键盘	屏幕键盘，可点按键发声
音乐	节奏训练	跟拍点击的节奏练习
音乐	校音器	麦克风校音
音乐	合成器	电子音色播放（多种波形）
交互	文本	显示一段文字（题面、反馈、进度……）
交互	按钮	玩家点击触发事件
交互	选择器	选项列表，玩家点选作答
交互	滑块	拖动取值
交互	输入框	文本输入
隐形	计时器	不显示在画面上，只按时间发信号

【关于「隐形」组件】隐形组件在画布上显示为半透明的幽灵框（方便你选中它），但在游玩页**完全不渲染**，只参与逻辑——典型的例子是计时器：到点发信号，画面上看不见它。

2.2 摆放与调整组件

对画布上的组件：

- **点选**：单击选中，右侧出现属性检查器；
- **移动**：按住拖动（画布内自动吸收、不会拖出边界）；
- **缩放**：拖右下角的手柄；
- **预览**：画布上渲染的是真实组件外观，但禁用交互——想真正点一点，用 「**试运行**」。

2.3 属性检查器

选中组件后，右侧面板可以设置：

- **名称**：组件的引用名（节点图里靠它找到这个组件，建议起有意义的名字，如 `question_label`、`answer_choice`）；
- **可见**：取消勾选后运行时隐藏（画布上显示为幽灵框）；
- **所属视图**：把组件挪到另一个视图；
- **X / Y / 宽 / 高**：位置与尺寸的精确数值；

- **属性设置**：每种组件自己的参数，用表单呈现。例如：按钮有「按钮文字」「初始可用」；滑块有「最小值 / 最大值 / 步长 / 初始值」；指法键盘有「最低音 / 最高音」；合成器有「音色」「音量」；五线谱有「音符可点击」；
- **props JSON（高级）**：以 JSON 形式查看/编辑上述属性，新手可不使用；
- **删除组件**：删除选中的组件。

2.4 视图：互斥的多幅画面

画布中间顶部是**视图条**。视图（view）就是「一幅独立的画面」：运行时**同一时刻只显示一个视图**，适合做「欢迎页 → 练习页 → 结算页」这样的流程切换。

- 新建关卡默认只有一个「主视图」；点 「+ **视图**」可添加（自动命名为视图 2、视图 3…）；
- 点击视图 chip 切换编辑对象（当前视图前有箭头标记）；
- chip 右侧 × 删除视图——**视图里的组件会被一并删除**；至少要保留一个视图；
- 想把组件挪到别的视图，用属性检查器里的「所属视图」下拉。

2.5 保存并试运行

点 「**保存**」保存文档；点 「**试运行**」会先保存再跳转游玩页。游玩页的构成：顶部是标题、当前题面与「第 N / M 题」进度；中间是关卡画面（你的组件）；底部是结算信息与 「**重开本关**」按钮。

> **【小练习】**新建一个关卡，在主视图里完成：添加一个**文本**组件写上欢迎语，再添加一个**按钮**，把按钮文字改成「开始」。保存后点 「**试运行**」，确认画面正常显示。现在点按钮不会发生任何事——因为还没写逻辑，这正是下一章的内容。

3 数据表：填入题目内容

3.1 一行就是一道题

切到 「**数据表**」标签页。这里的核心约定非常简单：

【数据表的三个约定】

1. **每一行是一道题**；
2. **行号从 0 开始**，当前行号保存在系统变量 `v.__row` 中；
3. 逻辑里用 `q.` 列名引用「当前行」的内容。比如第 2 行正在出题时，`q.title` 就是第 2 行「title」列的值。

空模板的数据表自带两列：`data`（JSON 高级）与 `scoring`（数字，默认 10，表示本题分值）。你可以直接使用、改名或删掉它们，按需搭自己的列。

3.2 列与列类型

数据表上方点 「＋ **添加行**」增加题目；在**列头**定义列：每列有「列名」（逻辑引用用，即 `q.列名`）和「显示名」（给人看的），以及一个**类型**。类型决定了单元格用什么控件编辑、以及逻辑里取出来的数据形状：

类型	单元格控件	适合放什么
文本	文本输入框	题面、答案、反馈文案
数字	数字输入框	分值、正确序号、目标值
布尔	勾选框	是/否标记
音符串	五线谱编辑器	要播放/展示的乐谱
字符串列表	条目列表（增删条目）	选择题的选项
JSON 高级	JSON 编辑弹层	复杂结构（兜底用）

【提示】列头可以拖右缘调宽，行头可以拖下缘调高。改列名后，逻辑里旧的 `q.旧列名` 引用会被 lint（静态检查）提示，跟着提示改即可。类型不符的旧数据会自动回落到「编辑 JSON」兜底入口。

3.3 五线谱编辑器

「音符串」列的单元格点开后是一个可视化五线谱编辑弹窗：可以直接**点谱线加音**，也支持一套接近 MuseScore 的键盘输入：

按键	作用
C–G	录入对应音名（自动落到最近的合适白键）
Shift（按住）	横向叠加，构成和弦
0	插入休止符
1–6	选择基础时值（数字越小音符越长）
.	附加附点（延长一半）
上/下方向键	升高 / 降低八度
＋ / －	升半音 / 降半音
左/右方向键	移动插入光标
Delete	删除光标处元素
空格	试听当前内容

3.4 推进与过关设置

数据表表格上方是**推进设置**，控制题目怎么出、怎样算过关：

- **顺序**：「**顺序出题**」（按行序）或 「**随机乱序**」；
- **本局题数**：留空表示全部行都出；填数字则本局只出 N 题（配合乱序可做「10 题池抽 5 题」）；
- **结算校验过关条件**：勾选后可用条件构造器写过关线。空模板默认是 `v.score >= 10`——得分达到 10 分才算通过。条件里引用了未声明的变量时会出现黄色警告。

> **【小练习】**把数据表改成你自己的：删掉或保留默认两列，新建三列——`title`（文本，题面）、`notes`（音符串，要听的音）、`answer`（文本，正确答案）。用 「**＋ 添加行**」做 3 道题，音符列用五线谱编辑器各录一个音。这份表就是第 4–5 章要用的素材。

4 节点图：让关卡活起来

前两章搭好了「画面」和「内容」，但现在的关卡是静止的：点按钮没反应，答对不加分。**节点图**负责回答「什么时刻、发生什么」。它的思路不是写代码，而是把一个个「节点卡」用线连成流程。

4.1 工作台导览

节点图标签页是一个可自由摆放大面板的工作台，顶部工具栏可以开关各个面板（布局会自动记住）：

按钮 / 面板	作用
「**组件库**」	节点库：按分组浏览所有可添加的节点，支持搜索
「**事件**」	事件管理器：本关所有可用的「信号源」一览
「**变量**」	变量面板：查看/新增/改名/删除变量
「**属性**」	属性检查器：编辑选中节点的参数
「**AI**」	AI 助手面板（与编辑器其他页共用会话）
「**调试**」	调试面板：断点、单步、实时变量（见 6.4 节）
「**整理布局**」	自动把节点排整齐
「**查看脚本**」	只读查看本图对应的伪代码
「**运行日志**」	最近一次运行的日志与错误（点错误可定位节点）
「**画布对照**」	右下角显示画布缩略图，点组件可高亮相关节点

画布基本操作（界面上也有提示）：「**右键画布空白**」加节点 · 「**拖端口**」连线（真/假出口） · 「**双击连线**」断开 · 「**Delete**」删除选中节点。

4.2 事件：一切逻辑的起点

打开「**事件**」面板，会看到本关所有可用的事件，按来源分组：

分组	事件	触发时机
关卡	`level.started`	进入关卡后触发一次（做初始化）
关卡	`level.finished`	结算时触发（负载带得分与通过状态）
题目	`question.loaded`	每行数据装载后触发（做本题的展示）
视图	`view.entered`	切换到某视图后触发
组件	各组件自己的事件	玩家操作了某个组件
内部	自定义（名: 名）	你自己发出、自己接收的信号

常用组件事件一览：按钮 `clicked`（被点击）；选择器 `chosen`（被点选，负载带选项序号与值）；滑块 `changed`（值变化）；输入框 `submitted`（提交）；五线谱 `noteClicked`（点了音符）；键盘 `keyClicked`（点了琴键）；计时器 `tick`（到点）。

每个事件行上的「＋ **处理器**」按钮会把一个紫色**事件节点**放到画布上——它就是一条流程的起点。

【空模板自带的半成品】新建关卡时，图里已经有一条链：`level:correct` → 分数加 10。`level:correct` 是一个约定的「答对」内部事件，但空关卡里**还没有人发出它**，所以这条链暂时不会触发——你需要在判定答对的地方用「触发内部事件」节点发出 `level:correct`（参考内置示例「听音点击」的做法），或者干脆删掉它，直接在自己的判断分支里加分。

4.3 连节点：把流程画出来

除了事件节点，常用的还有：

节点	颜色	作用
动作（call）	蓝	指挥某个组件或系统做事
赋值（assign）	绿	给变量赋值
条件分支	橙	按条件走「真 / 假」两条出口
循环	橙	重复执行一段流程（固定次数或按条件）
等待	橙	停顿指定毫秒
触发内部事件	黄	发出一个自定义信号
注释	灰	写备注，不参与运行

连线规则对新手非常友好：

- 流程**从上往下**连：把上一个节点的出口拖到下一个节点的入口；

- **事件节点没有入口**（它本来就是起点），其他节点都有入口；
- 条件分支和循环有两个出口，标着**真 / 假**（真出口绿色、假出口红色），普通出口是蓝灰色；
- 没有独立的「数据线」——所有数据（引用哪个列、比较什么值）都**内联写在节点参数里**。

【不想从零画？用模板】节点库顶部的「模板」分组提供两条预连好的常用链，一键放入画布：「**计分初始化与门槛结算**」（关卡开始清零分数、结算时按过关线处理）与 「**防卡死守卫**」（当画布里有计时器组件时出现，防止逻辑卡死后无法结算）。

4.4 动作节点：指挥组件做事

选中一个蓝色动作节点，右侧属性面板依次选三样东西：

1. **实例**：指挥谁？下拉里有系统对象（关卡 level / 题目 question / 视图 views）和你画布上的每个组件（按「名称（id）」列出）；
2. **动作**：做什么？下拉按所选实例列出可用命令，每项附灰色说明文字；
3. **参数**：怎么做？多数参数是表单；发声类命令还提供音符可视化编辑。

常用的系统动作只有几个，都很好记：「**进入下一题**」（`question.next`，推进到下一行，末题自动结算）、「**重开本关**」（`level.restart`）、「**结算关卡**」（`level.finish`）、「**前往视图**」（`views.goto`，下拉选目标视图）。

【三条「把当前行刷到画面上」的命令】这是数据驱动玩法的核心。在 `question.loaded` 事件后面接动作节点，把当前行的内容应用到组件上：

命令	作用对象	从数据表取哪类列
`setText`	文本组件（改文字）	文本 / 数字 / 布尔
`setOptions`	选择器（改选项）	字符串列表
`setMusic`	五线谱（改乐谱）	音符串 / JSON

配置参数时会出现**数据表列下拉**，每项形如「显示名（列名）• 类型」，点选即可，完全不用手写表达式；特殊需求可切到「高级表达式」模式。列的类型与命令不匹配时会有黄色警告。

4.5 变量与计分

打开 「**变量**」面板：列表首行是系统变量 `__row`（当前题目行，只读）；点新增可创建自己的变量（默认初值 0），可改名、可删除。以 `__` 开头的系统变量不开放编辑。

绿色**赋值节点**给变量赋值。值的类型可以切换：数字 / 文本 / 开关（布尔）/ 变量或路径 / 查询（读取组件当前值，如滑块的值）/ 表达式。例如「答对加 10 分」就是：

```
v.score = v.score + 10
```

4.6 保存、检查、试运行

逻辑写完回到「**保存**」。若节点图有问题，工具栏会出现红色「**N 个问题**」按钮，点开是问题清单，**点任意一条可直接定位到对应节点**（比如引用了不存在的列、类型不匹配、连线断裂）。改完后 「**试运行**」实测一遍。

> **【小练习】**继续第 3 章小练习的关卡：(1) 在 `question.loaded` 后接三个动作——文本组件 `setText` 选 `title` 列、选择器 `setOptions` 选 `options` 列（可再加一列字符串列表）、发声组件 `play` 选 `notes` 列；(2) 加「答对加分」：在选择器 `chosen` 事件后接条件分支，判断 `event.value == q.answer`，真出口接「分数加 10」赋值与 「**进入下一题**」；(3) 保存并试运行——一个能玩、能计分、能过关的听音选择题就诞生了。

5 实战：两个完整案例

本章把前三章串成两个可直接套用的关卡形态。做完它们，大多数练习题都能变着花样做出来。

5.1 案例一：听音选择题（数据驱动）

目标：每题播放一个音或一段旋律，玩家从选项中选出答案；答对加分，全部结束按总分结算。

第一步：设计数据表。建议的列：

列名	类型	内容
`title`	文本	题面（如「这是什么音?」）
`notes`	音符串	要播放的音
`options`	字符串列表	选项（如 Do / Re / Mi）
`answer`	文本	正确选项的文本
`scoring`	数字	本题分值

第二步：搭画布（都在主视图）：一个**文本**组件显示题面；一个**发声器**或**合成器**播放 `notes`；一个**选择器**展示选项；一个**文本**组件作为反馈区；可加一个**按钮**「重播」。

第三步：画节点图：

1. 放入模板 「**计分初始化与门槛结算**」；

2. question.loaded → setText（题面） → setOptions（选项） → setMusic 或发声命令（播放 q.notes）；
3. 选择器 chosen → 条件分支 event.value == q.answer：真出口 → 加分（v.score = v.score + q.scoring） → 反馈文本 show「答对了」（语气选 success） → 「**进入下一题**」；假出口 → 反馈文本 show「再想想」（语气选 error），停留本题；
4. 「重播」按钮 clicked → 发声命令再播 q.notes。

> **【防重复计分】**chosen 在玩家点选后触发；若想杜绝反复点击刷分，可以在真出口加一把「锁」：先判断 v.answered == 0，加分后立刻 v.answered = 1，并在 question.loaded 里把它清回 0。另外可对选择器执行 reveal 命令（参数为正确项序号），界面会高亮正确项并锁定选项。

体验数据驱动的威力：想从 3 题扩到 20 题？数据表加 17 行，逻辑一个节点都不用动。

5.2 案例二：多视图闯关流程（视图驱动）

目标：欢迎页 → 练习页 → 结算页的完整流程，结算页可「再学一遍」。

第一步：搭三个视图：「+ 视图」两次，把三个视图分别布置好——欢迎页放引导文字和「开始」按钮；练习页放题目相关组件（可沿用案例一的全部逻辑）；结算页放结果文字和「再学一遍」按钮。

第二步：连线：

1. level.started → 「**前往视图**」（欢迎页）；
2. 欢迎页「开始」按钮 clicked → 「**前往视图**」（练习页）；
3. 练习页逻辑照案例一配置（题面刷新靠 question.loaded，**换题不换视图**）；
4. level.finished → 按得分给结算页文本赋文案 → 「**前往视图**」（结算页）；
5. 结算页「再学一遍」按钮 clicked → 「**重开本关**」（变量与组件状态全部复位，回到 level.started）。

内置关卡「新手引导 • 交互教程」就是这一形态的完整范例（五个视图分步教学），拿不准细节时直接看它的节点图。

5.3 两种形态怎么选

	数据驱动（案例一）	视图驱动（案例二）
画面	一个主视图，内容随行刷新	多个视图手动切换
适合	题库型练习（刷题、听辨）	流程型内容（引导、闯关）
加题成本	加一行数据	视情况（练习区随行刷新）
参考示例	音色实验室	新手引导

两种形态可以混用：欢迎/结算用视图切换，中间练习区交给数据表。

6 进阶技巧

6.1 条件、循环、等待与自定义事件

- **条件分支**：属性面板提供条件构造器——左操作数、比较符、右操作数三段式（也可切换真值判断或高级表达式）。例如「滑块值等于 42」：`event.value == 42`；
- **循环**：两种模式——重复固定次数、满足条件时循环；
- **等待**：停顿指定毫秒，适合做「亮一下再消失」之类的时间效果；
- **触发内部事件**：发出一个「名: 名」格式的自定义信号（如 `app:hint`），图中对应的事件节点会响应它。适合把一段流程抽出来复用，或在多个地方触发同一段逻辑（前文的 `level:correct` 就是这个机制）。

6.2 表达式速成

节点参数里的值有六种形态：数字、文本、开关（布尔）、变量或路径、查询、表达式。常用的引用写法：

写法	含义
`v.score`	变量
`q.title`	数据表当前行的列
`event.value`	刚触发事件的负载字段（随事件不同而不同）
`v.__row`	当前题目行号（系统变量）
`slider1.getValue()`	查询：读取组件当前值

常用比较符：==（相等）、!=、>、>=、<、<=；可用 +、- 等做算术。**注意 == 是严格相等**：数字 10 与文本"10" 不相等，判断前先确认列类型。

【提示】记不住写法也没关系：几乎所有表达式输入框都支持下拉候选（数据表列会出现在候选池里），先选后改，比手写快且不易错。

6.3 计时器与隐形组件

计时器组件（隐形）有两个命令：`start`（参数：毫秒数，可选重复）与 `stop`；到点发出 `tick` 事件。典型用法：开始答题时启动 10 秒计时，`tick` 触发后执行「时间到」的分支（提示、扣分或强制结算）。注意隐形组件**不会因为切视图而停止**——不需要时要显式 `stop`，画布含计时器时节点库里的「防卡死守卫」模板值得一用。

6.4 调试模式：看得见的排错

写复杂逻辑时，「**试运行**」只能看结果，**调试模式**能看过程。在节点图工具栏打开「**调试**」面板，点 「**启动调试**」：

- 工作台切换为调试布局：**左侧是真实可玩的游玩画面**（你的点击会作为真实事件进入调试会话），右侧是调试面板与 AI；
- **断点**：点节点卡左侧的小圆点设为断点，运行到此暂停；暂停时画布自动定位并高亮挂起的节点；
- **控制**：继续 / 单步 / 暂停 / 停止；
- **实时变量**：查看所有变量当前值，也可以直接改值再继续——改一个临界值试试分支走向，非常直观；
- **手动触发事件**：不点界面也能派发事件（可选事件、填负载），用来复现「如果玩家点了选项会发生什么」；
- 点 「**停止调试**」结束并恢复编辑布局。

6.5 排错工具箱

- **lint 问题面板**（红色「N 个问题」按钮）：保存前的静态检查，点条目定位节点；
- **运行日志**：最近一次运行的日志与错误，同样可点击定位；
- **画布对照**：右下角缩略图，点画布组件高亮图中与它相关的节点——「这个按钮都连了什么」一目了然；
- **查看脚本**：以伪代码形式只读预览整张图（见 7.1 节）。

7 高级：脚本、JSON 与 AI 协作

7.1 脚本页：用文字写逻辑

「**脚本**」标签页展示与节点图**完全等价的伪代码**（C/JS 大括号风格），与图一一对应、双向无损：

```
on level.started {
```

```
    v.score = 0;
}

on question.loaded {
    question_label.setText(q.title);
    answer_choice.setOptions(q.options);
    voice.play(q.notes);
}

on answer_choice.chosen {
    if (event.value == q.answer) {
        v.score = v.score + q.scoring;
        question.next();
    }
}
```

两个按钮构成工作流：「**应用到节点图**」（按脚本重建整张图，变量保留；解析失败会精确报出「第 X 行第 Y 列」且原文档不受影响）与「**从节点图重新生成**」（把图翻译成文字）。若图在脚本页之外被修改过，会出现黄色脏提示条，让你选择对齐或覆盖。

> **【提示】**变量不在脚本里——它们由变量面板单独管理。对新手而言，脚本页更适合**阅读与复查**（比大片节点直观），修改仍建议在节点图完成；复杂图若超出伪代码能表达的结构，页面会明确提示并建议改用节点图或 JSON。

7.2 JSON 页：关卡的原始形态

「**JSON**」页直接展示关卡文档全文，可编辑后点「**应用 JSON 到编辑器**」。两条硬约束：文档 `kind` 必须是 level；关卡 `id` 被强制保留（改了不生效）。应用前会自动做一次格式归一化，防止旧格式混入。适合做**整关备份**或把一段结构复制给另一关。

7.3 AI 助手：你的创作副驾驶

先配置：「**设置**」页添加供应商——填名称、API 地址（OpenAI 兼容格式，通常以 /v1 结尾）、模型名与 API Key（仅保存在本机浏览器），点「**测试连接**」确认后保存。

编辑器里点标签栏「**AI**」打开面板，三种模式按「放手程度」递增：

模式	行为	适合
ask	只读问答，不改文档	问概念、求诊断
plan	先产出编号计划，确认后执行	大改前先对齐思路
agent	直接修改关卡	让 AI 动手，改前自动存检查点，可一键回滚

输入区上方有三个快捷指令：「**审查关卡**」（输出问题/建议/亮点/总结并标注问题节点）、「**批量出题**」（按现有列结构追加 5 道题）、「**生成介绍**」（为关卡写一段介绍文案）。还支持粘贴截图（最多 4 张）让 AI「看图说话」；会话按关卡分别保存，下次打开继续聊。

【AI 也会错】agent 模式改完务必自己 「**试运行**」验证；不满意就 「**回滚**」到检查点。AI 的每次修改与手动编辑共用同一套撤销栈（Ctrl+Z 也能撤销 AI 的改动）。

7.4 导入、导出与分享

资源库中任意关卡行可 「**导出**」：单个关卡是 `.json` 文件；整个系列会打包成 `.zip`（内含清单与全部资源）。资源库顶部 「**导入**」接受这两种文件，完成后报告「新增 / 更新 / 跳过 / 拒绝」数量；若包里缺了被引用的文件会明确提示。想分享练习给朋友或学生：导出文件，对方导入即可，全程不需要服务器。

8 附录

附录 A ·组件速查表

组件	事件	常用命令	备注
文本	—	`setText`、`show`（可带语气）	题面、反馈首选
按钮	`clicked`	`setText`、`setEnabled`	流程推进常用
选择器	`chosen`	`setOptions`、`reveal`、`reset`	`reveal` 高亮正确项并锁定
五线谱	`noteClicked`	`setMusic`、`highlight`、`clear`	可配置音符可点击
发声器	—	`play`（音符串/速度）、`stop`	采样音色（钢琴等）
合成器	—	`play`（音符串/波形等）、`setWave`、`stop`	电子音色
指法 / 键盘	`keyClicked`	`highlight`、`clear`	属性可设音域
滑块	`changed`	`setValue`	可查询当前值
输入框	`submitted`	`setValue`、`clear`	提交时触发
计时器	`tick`	`start`（毫秒/重复）、`stop`	隐形组件
节奏训练	`tap`、`roundDone`	`start`（bpm/拍数）、`stop`	跟拍练习
校音器	`pitch`（约 10Hz）	`start`（需麦克风）、`stop`	隐形组件

所有组件另有两个通用命令：`setVisible`（显示/隐藏）与 `setEnabled`（启用/禁用）。

附录 B ・节点类型速查表

节点	参数要点	说明
事件（on）	选择事件	唯一起点，无入口
动作（call）	实例 → 动作 → 参数	指挥组件或系统
赋值（assign）	变量 + 值	值支持六种形态
条件分支	条件	真 / 假双出口
循环	次数或条件	双出口
等待	毫秒	停顿
触发内部事件	事件名（名: 名）+ 负载	发自定义信号
注释	文本	不参与运行

常用系统动作：`question.next`（进入下一题）、`level.restart`（重开本关）、`level.finish`（结算关卡）、`views.goto`（前往视图）、`v.__row` 赋值（跳转到指定题目行）。

附录 C ・列类型与命令匹配表

命令	接受的列类型	说明
`setText`	文本 / 数字 / 布尔	数字与布尔自动转文本
`setOptions`	字符串列表	每个条目即一个选项
`setMusic`	音符串 / JSON	直接取整列乐谱

类型不匹配时：静态检查报错或警告、下拉选择处黄色提示、运行时做宽松转换并在日志提示——三层防线，越早发现越好改。

附录 D ・常见问题（FAQ)

保存出现红条？

按提示清单逐条修正。常见原因：JSON 高级单元格语法错误、必填信息缺失。

出现黄色 lint 提示？

多为引用了不存在的列或类型不匹配。点工具栏「N 个问题」直接定位。

编辑内置示例后保存提示「另存为副本」?

正常保护机制：内置示例不可覆盖，你的修改会存成自己的副本。

组件在游玩页不见了？

两种可能：属性里「可见」被取消（画布上呈幽灵框）；或它属于另一个视图（当前视图不渲染）。

试运行没有声音？

先到设置页用「主音量 • 试听」排查设备；再检查组件的音量属性；部分浏览器要求先与页面交互一次才允许发声。

表达式不生效？

检查 `q.` 列名拼写（改过列名会有 lint 提示）、确认 `==` 两边类型一致（数字 vs 文本）。

调试启动后停住不动？

多半停在断点上：点「继续」，或不再需要时「清空」断点列表。

加了行但游玩时题数不对？

检查推进设置的「本局题数」是否填了数字（留空才是全部行）。

附录 E · 下一步

按这个顺序继续深入最顺：**改数据**（第 3 章） → **改画面**（第 2 章） → **改逻辑**（第 4–6 章） → **读懂内置示例的图与表**（资源库教程与示例分区） → **让 AI 当助手**（第 7 章）。卡住的时候，内置示例的节点图永远是最好的参考书。

本文基于 2026 年 10 月的编辑器界面与行为撰写；若界面后续更新，请以实际软件为准。